



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)

Кафедра инструментального и прикладного программного обеспечения (ИиППО)

ОТЧЁТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №3
по дисциплине «Программирование на языке Джава»

Выполнил студент группы **ИКБО-50-25**

Багинян А.В.

Принял
Старший преподаватель

Ермаков С.Р.

Москва 2026

1. Задачи к выполнению

1.1. Часть 1

1.1.1. Задача 1

Ваша программа должна быть организована по пакетам:

- Пакет `vehicles` для классов `Vehicle`, `Car`, `ElectricCar`, и интерфейса `ElectricVehicle`.
- Пакет `app` для тестового класса `TestCar`.

Создайте абстрактный класс `Vehicle`, который будет представлять общие характеристики любого транспортного средства. Включите следующие поля: `model`, `license`, `color`, `year`, `ownerName`, `insuranceNumber`, `engineType`. Определите методы: геттеры и сеттеры для каждого поля, а также метод `toString()`, который возвращает строку с описанием транспортного средства. Добавьте абстрактный метод `vehicleType()`, который будет возвращать тип транспортного средства (например, «Car», «Electric Car»). Класс `Car` должен наследовать абстрактный класс `Vehicle`. В конструкторе задавайте тип двигателя как «Combustion». Реализуйте метод `vehicleType()`, который возвращает «Car».

Определите интерфейс `ElectricVehicle`, который будет описывать специфические методы для электромобилей. В интерфейсе должны быть следующие методы: `getBatteryCapacity()` и `setBatteryCapacity()`. Класс `ElectricCar` должен наследовать класс `Car` и реализовывать интерфейс `ElectricVehicle`. Реализуйте методы интерфейса и добавьте поле `batteryCapacity` для хранения информации о ёмкости батареи. В конструкторе задайте тип двигателя как «Electric».

Требования к тестированию программы:

- Созданы экземпляры классов `Car` и `ElectricCar`.
- Изменение года выпуска и имени владельца.
- Изменение страхового номера.
- Получение информации о ёмкости батареи у электромобиля.
- Вывод информации о транспортных средствах в консоль с помощью метода `toString()`.

1.1.1.1. Решение

```
1 // vehicles/Vehicle.java
2 package ru.haykor.mirea.vehicles;
3
```

```

4 public abstract class Vehicle {
5     private String mModel;
6     private String mLicense;
7     private String mColor;
8     private int mYear;
9     private String mOwnerName;
10    private String mInsuranceNumber;
11    protected String mEngineType;
12
13    public Vehicle() {
14    }
15
16    public Vehicle(String model, String license, String color,
17    int year,
18                    String ownerName, String insuranceNumber,
19    String engineType) {
20        mModel = model;
21        mLicense = license;
22        mColor = color;
23        mYear = year;
24        mOwnerName = ownerName;
25        mInsuranceNumber = insuranceNumber;
26        mEngineType = engineType;
27    }
28
29    public abstract String vehicleType();
30
31    public String getModel() {
32        return mModel;
33    }
34
35    public void setModel(String model) {
36        mModel = model;
37    }
38
39    public String getLicense() {
40        return mLicense;
41    }
42
43    public void setLicense(String license) {
44        mLicense = license;
45    }
46

```

```

45     public String getColor() {
46         return mColor;
47     }
48
49     public void setColor(String color) {
50         mColor = color;
51     }
52
53     public int getYear() {
54         return mYear;
55     }
56
57     public void setYear(int year) {
58         mYear = year;
59     }
60
61     public String getOwnerName() {
62         return mOwnerName;
63     }
64
65     public void setOwnerName(String ownerName) {
66         mOwnerName = ownerName;
67     }
68
69     public String getInsuranceNumber() {
70         return mInsuranceNumber;
71     }
72
73     public void setInsuranceNumber(String insuranceNumber) {
74         mInsuranceNumber = insuranceNumber;
75     }
76
77     public String getEngineType() {
78         return mEngineType;
79     }
80
81     public void setEngineType(String engineType) {
82         mEngineType = engineType;
83     }
84
85     @Override
86     public String toString() {
87         return "Type=" + vehicleType() +

```

```

88         ", Model='" + mModel + '\\'' +
89         ", License='" + mLicense + '\\'' +
90         ", Color='" + mColor + '\\'' +
91         ", Year=" + mYear +
92         ", Owner='" + mOwnerName + '\\'' +
93         ", Insurance='" + mInsuranceNumber + '\\'' +
94         ", EngineType='" + mEngineType + '\\'';
95     }
96 }

```

```

1 // vehicles/Car.java
2 package ru.haykor.mirea.vehicles;
3
4 public class Car extends Vehicle {
5
6     public Car() {
7         super();
8         mEngineType = "Combustion";
9     }
10
11     public Car(String model, String license, String color, int
year,
12                String ownerName, String insuranceNumber) {
13         super(model, license, color, year, ownerName,
insuranceNumber, "Combustion");
14     }
15
16     @Override
17     public String vehicleType() {
18         return "Car";
19     }
20
21     @Override
22     public String toString() {
23         return "Car{" + super.toString() + "}";
24     }
25 }

```

```

1 // vehicles/ElectricVehicle.java
2 package ru.haykor.mirea.vehicles;
3
4 public interface ElectricVehicle {

```

```
5     double getBatteryCapacity();
6
7     void setBatteryCapacity(double batteryCapacity);
8 }
```

```
1 // vehicles/ElectricCar.java
2 package ru.haykor.mirea.vehicles;
3
4 public class ElectricCar extends Car implements ElectricVehicle
5 {
6     private double mBatteryCapacity;
7
8     public ElectricCar() {
9         super();
10        mEngineType = "Electric";
11    }
12
13    public ElectricCar(String model, String license, String
14    color, int year,
15                        String ownerName, String
16    insuranceNumber, double batteryCapacity) {
17        super(model, license, color, year, ownerName,
18    insuranceNumber);
19        mEngineType = "Electric";
20        mBatteryCapacity = batteryCapacity;
21    }
22
23    @Override
24    public String vehicleType() {
25        return "Electric Car";
26    }
27
28    @Override
29    public double getBatteryCapacity() {
30        return mBatteryCapacity;
31    }
32
33    @Override
34    public void setBatteryCapacity(double batteryCapacity) {
35        mBatteryCapacity = batteryCapacity;
36    }
37}
```

```

35     public String toString() {
36         return "ElectricCar{" + super.toString() + ",
BatteryCapacity=" + mBatteryCapacity + " kWh}";
37     }
38 }

```

```

1 // app/TestCar.java
2 package ru.haykor.mirea.app;
3
4 import ru.haykor.mirea.vehicles.Car;
5 import ru.haykor.mirea.vehicles.ElectricCar;
6 import ru.haykor.mirea.vehicles.Vehicle;
7
8 public class TestCar {
9     public static void main(String[] args) {
10         Vehicle car = new Car("BMW 3", "A100AA77", "Black",
2020, "Alex", "INS-111");
11         Vehicle electricCar = new ElectricCar("Tesla Model 3",
"B200BB77", "White", 2022, "John", "INS-222", 75.0);
12
13         car.setYear(2021);
14         car.setOwnerName("Dmitry");
15         car.setInsuranceNumber("INS-333");
16
17         electricCar.setYear(2023);
18         electricCar.setOwnerName("Ivan");
19         electricCar.setInsuranceNumber("INS-444");
20
21         ElectricCar tesla = (ElectricCar) electricCar;
22         System.out.println("Battery capacity: " +
tesla.getBatteryCapacity() + " kWh");
23
24         System.out.println(car);
25         System.out.println(electricCar);
26     }
27 }

```

1.2. Часть 2

1.2.1. Задача 1

Проектирование интерфейсов в классах Java. За основу взят проект `bikeproject`, состоящий из классов `Bike`, `MountainBike`, `RoadBike` и драйвера `BikeDriver`.

1.2.1.1. Ответы на вопросы

- Приведите пример примитивного типа данных, который используется для хранения полей в классе. Поле `numGears` в классе `Bike` и поле `frameSize` в классе `MountainBike` имеют тип `int`.
- Приведите пример того, где происходит конкатенация строк. В методе `Bike.printDescription()` при построении строки описания: `MAKE + "\n" + "This bike has " + handleBars`.
- Каковы имена объектов, созданных в этой программе? В классе `BikeDriver` созданы объекты `bike1`, `bike2`, `bike3`, `bike4`.
- Сколько конструкторов у каждого класса? `Bike` — 2 конструктора; `MountainBike` — 2 конструктора; `RoadBike` — 3 конструктора; `BikeDriver` — явных конструкторов нет (используется конструктор по умолчанию).
- Определите супер- и подклассы из этой программы. Суперкласс — `Bike`. Подклассы — `MountainBike` и `RoadBike`, оба наследуют `Bike`.
- Значения полей стандартных (созданных по умолчанию) велосипедов каждого типа приведены в (см. Таблица 1.1).

Таблица 1.1 — Значения полей стандартных велосипедов по умолчанию

Поле	MountainBike (по умолчанию)	RoadBike (по умолчанию)
<code>handleBars</code>	Bull Horn	drop
<code>frame</code>	Hardtail	racing
<code>tyres</code>	Maxxis	tread less
<code>seatType</code>	dropper	razor
<code>numGears</code>	27	19
<code>suspension</code>	RockShox XC32	—
<code>type</code>	Pro	—
<code>frameSize</code>	19	—
<code>tyreWidth</code>	—	20
<code>postHeight</code>	—	22

1.2.1.2. Модернизация программы

1. Создайте интерфейс в проекте велосипеда, который устанавливает название компании-производителя велосипедов как неизменяемое значение. Он также определяет методы, которые должны быть реализованы любым классом, использующим интерфейс.
2. Создайте интерфейс с именем `MountainParts`, который имеет константу с именем `TERRAIN`, которая будет хранить строковое значение «off_road». Интерфейс определит два метода, которые принимают аргумент с именем `newValue`, и два метода, которые будут возвращать текущее значение поля экземпляра. Методы должны быть названы: `getSuspension`, `setSuspension`, `getType`, `setType`.
3. Создайте интерфейс `RoadParts`, который имеет константу с именем `terrain`, которая будет хранить строковое значение «track_racing». Интерфейс определит два метода, которые принимают аргумент с именем `newValue`, и два метода, которые будут возвращать текущее значение поля экземпляра. Методы должны быть названы: `getTyreWidth`, `setTyreWidth`, `getPostHeight`, `setPostHeight`.
4. Используйте интерфейс `BikeParts` с классом `Bike`, добавляя любые необходимые нереализованные методы. Добавьте требуемый внутренний код для каждого из добавленных методов.
5. Используйте интерфейс `MountainParts` с классом `MountainBike`, добавив все необходимые нереализованные методы. Добавьте требуемый внутренний код для каждого из добавленных методов.
6. Используйте интерфейс `RoadParts` с классом `RoadBike`, добавив все необходимые нереализованные методы. Добавьте требуемый внутренний код для каждого из добавленных методов.
7. Запустите и протестируйте свою программу, она должна работать точно так же, как и раньше.
8. В нижней части класса драйвера обновите высоту столба для `bike1` до 20 вместо 22.
9. Выведите значения `bike1` на экран, чтобы подтвердить изменение.
10. Запустите и протестируйте свою программу.

1.2.1.3. Решение

```
1 // bikeproject/BikeParts.java
2 package ru.haykor.mirea.bikeproject;
3
4 public interface BikeParts {
```

```
5     String MAKE = "Oracle Cycles";
6
7     void printDescription();
8 }
```

```
1 // bikeproject/MountainParts.java
2 package ru.haykor.mirea.bikeproject;
3
4 public interface MountainParts {
5     String TERRAIN = "off_road";
6
7     void setSuspension(String newValue);
8
9     void setType(String newValue);
10
11     String getSuspension();
12
13     String getType();
14 }
```

```
1 // bikeproject/RoadParts.java
2 package ru.haykor.mirea.bikeproject;
3
4 public interface RoadParts {
5     String terrain = "track_racing";
6
7     void setTyreWidth(int newValue);
8
9     void setPostHeight(int newValue);
10
11     int getTyreWidth();
12
13     int getPostHeight();
14 }
```

```
1 // bikeproject/Bike.java
2 package ru.haykor.mirea.bikeproject;
3
4 public class Bike implements BikeParts {
5
6     private String handleBars, frame, tyres, seatType;
7     private int numGears;
```

```

8
9     public Bike() {
10    }
11
12    public Bike(String handleBars, String frame, String tyres,
String seatType, int numGears) {
13        this.handleBars = handleBars;
14        this.frame = frame;
15        this.tyres = tyres;
16        this.seatType = seatType;
17        this.numGears = numGears;
18    }
19
20    @Override
21    public void printDescription() {
22        System.out.println("\n" + MAKE + "\n"
23            + "This bike has " + this.handleBars + "
handlebars on a "
24            + this.frame + " frame with " + this.numGears
+ " gears."
25            + "\nIt has a " + this.seatType + " seat with
" + this.tyres + " tyres.");
26    }
27 }

```

```

1 // bikeproject/MountainBike.java
2 package ru.haykor.mirea.bikeproject;
3
4 public class MountainBike extends Bike implements MountainParts
{
5
6     private String suspension, type;
7     private int frameSize;
8
9     public MountainBike() {
10        this("Bull Horn", "Hardtail", "Maxxis", "dropper", 27,
"RockShox XC32", "Pro", 19);
11    }
12
13    public MountainBike(String handleBars, String frame,
String tyres, String seatType, int numGears,
14        String suspension, String type, int
frameSize) {

```

```

15         super(handleBars, frame, tyres, seatType, numGears);
16         this.suspension = suspension;
17         this.type = type;
18         this.frameSize = frameSize;
19     }
20
21     @Override
22     public void printDescription() {
23         super.printDescription();
24         System.out.println("This mountain bike is a " +
25 this.type + " bike and has a "
26         + this.suspension + " suspension and a frame
27 size of " + this.frameSize + " inches.");
28     }
29
30     @Override
31     public void setSuspension(String newValue) {
32         this.suspension = newValue;
33     }
34
35     @Override
36     public String getSuspension() {
37         return this.suspension;
38     }
39
40     @Override
41     public void setType(String newValue) {
42         this.type = newValue;
43     }
44
45     @Override
46     public String getType() {
47         return this.type;
48     }
49 }

```

```

1 // bikeproject/RoadBike.java
2 package ru.haykor.mirea.bikeproject;
3
4 public class RoadBike extends Bike implements RoadParts {
5
6     private int tyreWidth, postHeight;
7

```

```

8     public RoadBike() {
9         this("drop", "racing", "tread less", "razor", 19, 20,
10        22);
11    }
12    public RoadBike(int postHeight) {
13        this("drop", "racing", "tread less", "razor", 19, 20,
14        postHeight);
15    }
16    public RoadBike(String handleBars, String frame, String
17    tyres, String seatType, int numGears,
18        int tyreWidth, int postHeight) {
19        super(handleBars, frame, tyres, seatType, numGears);
20        this.tyreWidth = tyreWidth;
21        this.postHeight = postHeight;
22    }
23    @Override
24    public void printDescription() {
25        super.printDescription();
26        System.out.println("This Roadbike bike has " +
27        this.tyreWidth + "mm tyres and a post height of " +
28        this.postHeight + ".");
29    }
30    @Override
31    public void setTyreWidth(int newValue) {
32        this.tyreWidth = newValue;
33    }
34    @Override
35    public int getTyreWidth() {
36        return this.tyreWidth;
37    }
38    @Override
39    public void setPostHeight(int newValue) {
40        this.postHeight = newValue;
41    }
42    @Override
43    public int getPostHeight() {

```

```
46         return this.postHeight;
47     }
48 }
```

```
1 // bikeproject/BikeDriver.java
2 package ru.haykor.mirea.bikeproject;
3
4 public class BikeDriver {
5
6     public static void main(String[] args) {
7
8         RoadBike bike1 = new RoadBike();
9         RoadBike bike2 = new RoadBike("drop", "tourer", "semi-
10 grip", "comfort", 14, 25, 18);
11         MountainBike bike3 = new MountainBike();
12         Bike bike4 = new Bike();
13
14         bike1.printDescription();
15         bike2.printDescription();
16         bike3.printDescription();
17         bike4.printDescription();
18
19         bike1.setPostHeight(20);
20         bike1.printDescription();
21     }
22 }
```